

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	G.Srivardhan	Reg. No.:	192425208
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Aim:

To implement the Job Scheduling with Deadlines problem using the Greedy approach to maximize total profit while ensuring each job is completed within its specified deadline.

Procedure:

1. Represent each job as a pair consisting of a deadline and a profit.
2. Sort all the given jobs in descending order based on their profit. This is the greedy strategy to ensure high-value jobs are prioritized.

3. Find the maximum deadline among all jobs to determine the total number of available time slots.
4. Create an array (or list) of boolean values to keep track of occupied time slots, initialized to False.
5. For each job in the sorted list:
 - a. Start from its deadline and search for the nearest available free slot moving backwards towards time unit 1.
 - b. If an empty slot is found, assign the job to that slot, mark the slot as occupied, and add the job's profit to the total profit.
 - c. If no slot is available before or at the deadline, skip the job.
6. Return the total number of jobs scheduled and the total accumulated profit.

Code:

```
def job_scheduling(n, deadlines, profits):
    # Create a list of jobs as (deadline, profit) tuples
    jobs = []
    for i in range(n):
        jobs.append((deadlines[i], profits[i]))

    # Sort jobs in descending order of profit (Greedy Step)
    jobs.sort(key=lambda x: x[1], reverse=True)

    # Find the maximum deadline to determine the number of available slots
    max_deadline = max(deadlines)

    # Initialize slots to keep track of filled time units (1-based indexing)
    slot = [False] * (max_deadline + 1)

    total_profit = 0
    jobs_done = 0

    # Iterate through all sorted jobs
    for d, p in jobs:
        # Try to find a free slot for this job, starting from its deadline moving backwards
        for t in range(d, 0, -1):
            if not slot[t]:
                # If a slot is found, mark it as filled
                slot[t] = True
                total_profit += p
                jobs_done += 1
                break

    return jobs_done, total_profit

# Driver Code for testing
if __name__ == "__main__":
    n = 4
    deadlines = [2, 1, 2, 1]
    profits = [100, 10, 20, 30]
```

```

jobs_done, total_profit = job_scheduling(n, deadlines, profits)
print("Number of jobs done:", jobs_done)
print("Total profit:", total_profit)

```

output

ASSESSMENT C04-AT1-Coding Challenge Task

Language: Python C C++ Java

QUESTIONS

Q1 Job Scheduling with Deadlines 100 marks

1/1 answered

Q1 Job Scheduling with Deadlines 100 marks

Each job has a deadline and a profit, and only one job can be scheduled at a time. The objective is to schedule jobs such that total profit is maximized while ensuring each job is completed before its deadline. Conflicts must be avoided, and optimal ordering is required. Use a Greedy strategy to determine the best schedule.

Input Format:

n = number of jobs
 deadline[] = deadlines
 profit[] = profits

Your Code

```

def job_scheduling(n, deadlines, profits):
    jobs = [(deadline[i], profit[i]) for i in range(n)]
    jobs.sort(key=lambda x: x[1], reverse=True)
    max_deadline = max(deadlines)
    slot = [False] * (max_deadline + 1)
    job_sequence = [-1] * (max_deadline + 1)
    total_profit = 0
    jobs_done = 0
    for d, p in jobs:
        for t in range(d, 0, -1):
            if not slot[t]:
                slot[t] = True
                job_sequence[t] = p
                total_profit += p
                jobs_done += 1
    return jobs_done, total_profit

```

Input

stdin input:

Output

Number of jobs done: 2
 Total profit: 130

Result:

The Job Scheduling algorithm was successfully implemented. For the input provided (Deadlines: [2, 1, 2, 1], Profits: [100, 10, 20, 30]), the program outputted:

Number of jobs done: 2

Total profit: 130

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION**Coding Challenge Task**

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				

Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____